PIMPRI-CHINCHWAD EDUCATION TRUST'S

Pimpri-Chinchwad College of Engineering

(An Autonomous Institute) • Affiliated to Savitribai Phule Pune University (SPPU) • ISO 21001:2018 Certified by TUV

ASSIGNMENT NO 3

Name:	**Siddhesh Parkar**	**Course:**	Design and Analysis of Algorithm
PRN:	**124B1F021**	**Class:**	TY IT A
Topic:	**Fractional Knapsack (Relief Supply Distribution)**	**Batch:**	A1

1. Problem Statement

Emergency Relief Supply Distribution: A devastating flood has struck multiple remote villages. A rescue team operates a limited-capacity boat that can carry a maximum weight of W kg. The boat must transport critical supplies (food, medicine, drinking water) from a relief base to affected settlements. Each item has a weight w_i and utility value v_i. Certain items are divisible (food packets, water), while others must be taken whole (medical kits). Implement the **Fractional Knapsack (Greedy Approach)** algorithm to maximize total utility within boat capacity W.

2. Course Objectives

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms such as Greedy methodologies.

4. Theory

The Fractional Knapsack problem is an optimization problem where items can be broken down into fractions. It satisfies two fundamental properties that prove greedy optimality:

- **Greedy-Choice Property:** Locally optimal choices (selecting items with the highest utility-to-weight ratio $r_i = v_i / w_i$) lead directly to a globally optimal solution.
- **Optimal Substructure:** An optimal solution contains within it optimal solutions to subproblems (after occupying partial capacity, remaining capacity is solved identically).

Complexity Analysis:

- **Ratio Computation:** Calculating ratios for n items takes O(n) time.
- **Sorting Phase:** Sorting n items by ratio in descending order takes **O(n log n)** time.
- **Greedy Selection:** Scanning sorted items and filling the knapsack takes O(n) time.
- **Overall Time Complexity: O(n log n)** | **Space Complexity:** O(n) for storing items.

5. Algorithm & Pseudo Code

```
function MaxUtilFractional(items, W):
    for each item i in items do
        item.ratio ← item.utility / item.weight
    end for
    Sort items in descending order of ratio
    totalUtility ← 0, remainingCapacity ← W
    for each item in sorted items do
        if remainingCapacity == 0 then break
        if item.weight ≤ remainingCapacity then
            totalUtility ← totalUtility + item.utility
            remainingCapacity ← remainingCapacity - item.weight
        else if item.divisible == "yes" then
            fraction ← remainingCapacity / item.weight
            totalUtility ← totalUtility + (item.utility * fraction)
            remainingCapacity ← 0
        end if
    end for
    return totalUtility
end function
```

6. Numerical Trace Example

Boat Capacity: W = 17 kg

Item	Weight (kg)	Utility	Ratio (Val/Wt)	Selection Status	Utility Earned
Medicine	5	50	10.0	FULL (5 kg)	50
Food	10	30	3.0	FULL (10 kg)	30
Water	20	20	1.0	PARTIAL (2 kg / 20 kg)	2

Total Weight Loaded: 5 + 10 + 2 = 17 kg • **Maximum Utility:** 50 + 30 + 2 = **82.00**

7. Program Source Code (C++)

```
/*
 * Student Name: Siddhesh Parkar
 * PRN: 124B1F021
 * GitHub Repository: https://github.com/siddhesh160606/DAA_lab
 * Assignment 3: Emergency Relief Supply Plan using Fractional Knapsack (Greedy Approach)
 */

#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <algorithm>
#include <iomanip>
using namespace std;

struct Item {
    int itemId;
    string name;
    string category;
    double weight;
    double utility;
    string divisible;
    double ratio;
    double selectedWeight;
    double selectedUtility;
};

// Compare items based on utility/weight ratio
bool compareRatio(const Item& a, const Item& b) {
    return a.ratio > b.ratio;
}

int main() {

    vector<Item> items;

    // Open CSV file
    ifstream file("relief_items.csv");

    if (!file) {
        cout << "Error: Could not open relief_items.csv" << endl;
        return 1;
    }

    string line;

    // Skip header
    getline(file, line);

    // Read dataset
    while (getline(file, line)) {

        stringstream ss(line);
        string value;
        Item item;

        // item_id
        getline(ss, value, ',');
        item.itemId = stoi(value);

        // name
        getline(ss, value, ',');
        item.name = value;

        // category
```

```
        getline(ss, value, ',');
        item.category = value;

        // weight_kg
        getline(ss, value, ',');
        item.weight = stod(value);

        // utility_value
        getline(ss, value, ',');
        item.utility = stod(value);

        // divisible
        getline(ss, value, ',');
        item.divisible = value;

        // Calculate utility/weight ratio
        item.ratio = item.utility / item.weight;

        item.selectedWeight = 0;
        item.selectedUtility = 0;

        items.push_back(item);
    }

    file.close();

    cout << "Student Name : Siddhesh Parkar\n";
    cout << "PRN          : 124B1F021\n";
    cout << "GitHub Link  : https://github.com/siddhesh160606/DAA_lab\n";
    cout << "-----------------------------------------------------\n";
    cout << "Total items read: " << items.size() << endl;

    // Display capacity
    double capacity;

    cout << "\nEnter boat capacity (kg): ";
    cin >> capacity;

    // Sort according to utility/weight ratio
    sort(items.begin(), items.end(), compareRatio);

    double remainingCapacity = capacity;
    double totalUtility = 0;

    // Select items
    for (auto& item : items) {

        if (remainingCapacity <= 0)
            break;

        // Divisible item
        if (item.divisible == "yes") {

            if (item.weight <= remainingCapacity) {

                // Take complete item
                item.selectedWeight = item.weight;
                item.selectedUtility = item.utility;

                remainingCapacity -= item.weight;
                totalUtility += item.utility;
            }
            else {

                // Take fraction of item
                double fraction = remainingCapacity / item.weight;

                item.selectedWeight = remainingCapacity;
```

```
                item.selectedUtility = item.utility * fraction;

                totalUtility += item.selectedUtility;

                remainingCapacity = 0;
            }
        }

        // Non-divisible item
        else {

            if (item.weight <= remainingCapacity) {

                // Must take complete item
                item.selectedWeight = item.weight;
                item.selectedUtility = item.utility;

                remainingCapacity -= item.weight;
                totalUtility += item.utility;
            }
        }
    }

    // Display results
    cout << fixed << setprecision(2);

    cout << "\n============================================\n";
    cout << "       EMERGENCY RELIEF SUPPLY PLAN\n";
    cout << "============================================\n";

    cout << "Boat Capacity       : " << capacity << " kg\n";
    cout << "Used Capacity       : "
         << capacity - remainingCapacity << " kg\n";
    cout << "Remaining Capacity  : "
         << remainingCapacity << " kg\n";
    cout << "Total Utility       : "
         << totalUtility << "\n";

    cout << "\nSelected Items:\n";
    cout << "--------------------------------------------\n";

    for (const auto& item : items) {

        if (item.selectedWeight > 0) {

            cout << "ID: " << item.itemId
                 << " | " << item.name
                 << " | Weight: " << item.selectedWeight << " kg"
                 << " | Utility: " << item.selectedUtility;

            if (item.selectedWeight < item.weight) {
                cout << " | PARTIAL";
            }
            else {
                cout << " | FULL";
            }

            cout << endl;
        }
    }

    cout << "============================================\n";

    return 0;
}
```

8. Sample Execution Output

```
=======================================================
Student Name: Siddhesh Parkar
PRN         : 124B1F021
GitHub Repo : https://github.com/siddhesh160606/DAA_lab
Assignment  : 3 - Emergency Relief Supply Plan using Fractional Knapsack
=======================================================

Total items read: 30

Enter boat capacity (kg): 50.00

===============================================
       EMERGENCY RELIEF SUPPLY PLAN
===============================================
Boat Capacity       : 50.00 kg
Used Capacity       : 45.10 kg
Remaining Capacity  : 4.90 kg
Total Utility       : 802.20

Selected Items:
-----------------------------------------------
ID: 30 | Oral Rehydration Salts #30 | Weight: 1.00 kg | Utility: 98.00 | FULL
ID: 3 | Oral Rehydration Salts #3 | Weight: 2.70 kg | Utility: 92.70 | FULL
ID: 18 | First Aid Kit #18 | Weight: 4.90 kg | Utility: 99.40 | FULL
ID: 10 | Loose Antibiotic Powder #10 | Weight: 5.20 kg | Utility: 94.00 | FULL
ID: 16 | Loose Antibiotic Powder #16 | Weight: 4.60 kg | Utility: 80.50 | FULL
ID: 14 | Painkillers (Bulk) #14 | Weight: 4.70 kg | Utility: 73.80 | FULL
ID: 25 | Loose Antibiotic Powder #25 | Weight: 5.90 kg | Utility: 80.20 | FULL
ID: 20 | Water Purification Tablets #20 | Weight: 8.10 kg | Utility: 94.60 | FULL
ID: 22 | Oral Rehydration Salts #22 | Weight: 8.00 kg | Utility: 89.00 | FULL
===============================================
```

9. Conclusion

The Fractional Knapsack Algorithm solves emergency logistics distribution with provable polynomial-time optimality (O(n log n)). By prioritizing critical relief goods based on utility-to-weight ratios, life-saving medicines and high-energy rations are dispatched first, and remainder capacity is filled with divisible survival provisions, minimizing lost utility.

GitHub Repository Link

Repository Link: https://github.com/siddhesh160606/DAA_lab